

TP

Objectif

Créer une application PHP orientée objet complète, en utilisant toutes les notions vues en cours : encapsulation, relations POO (association, agrégation, composition), héritage, classes abstraites, interfaces, traits, méthodes statiques, PSR-4, namespaces, autoloading. Découvrir en autonomie : injection de dépendances, gestion d'exceptions, enums, évolution vers une gestion en base de données.

Éléments à modéliser

1) Auteur

- Propriétés : nom (obligatoire) + autres (à déterminer)
- Relation : un auteur peut avoir plusieurs livres (agrégation)
- Héritage : doit étendre la classe abstraite Personne
- Getters/setters obligatoires

2) Catégorie

- Propriétés : libellé (obligatoire) + autres (à déterminer)
- Relation : une catégorie regroupe plusieurs livres (agrégation)
- Possibilité d'utiliser une enum pour représenter certaines catégories fixes
- Getters/setters obligatoires

3) Livre

- Propriétés : titre, Auteur, Catégorie (obligatoires) + autres (à déterminer)
- Relations : association avec Auteur et Catégorie
- Méthodes : getDescription(), méthode statique optionnelle (factory ou compteur)
- Getters/setters obligatoires

4) Bibliothèque

- Propriétés : nom, collection d'ExemplaireLivre (obligatoire) + autres (à déterminer)
- Méthodes : ajouterLivre(), rechercherParAuteur(), rechercherParCategorie(), afficherTousLesLivresDisponibles()
- Logger obligatoire pour chaque ajout/emprunt
- Injection de dépendances : le logger doit être injecté via interface
- Relation : composition avec ExemplaireLivre (la bibliothèque crée et détruit ses exemplaires)

5) ExemplaireLivre (composition)

- Propriétés : livre, estEmprunte (obligatoire) + autres (à déterminer)
- Constructeur privé/protégé pour forcer la composition
- Méthodes : marquerEmprunte(), marquerRendu()
- Getters

6) Personne (classe abstraite)

- Propriété : nom (obligatoire) + autres (à déterminer)
- Méthode abstraite getType()
- Getter/setter

7) Utilisateur (hérite de Personne)

- Propriétés : email, liste d'emprunts (obligatoire) + autres (à déterminer)
- Méthodes : emprunter(), afficherLivresEmpruntés()
- Associations : Bibliothèque et ExemplaireLivre
- Logger obligatoire lors des emprunts
- Exceptions : lever une exception si aucun exemplaire n'est disponible
- Gestion via try/catch dans index.php
- Getters/setters obligatoires

8) Logger

- Interface LoggerInterface avec méthode log()
- Trait LoggerTrait réutilisable
- Injection de dépendances obligatoire : le logger ne doit pas être instancié dans les classes

Autoloading / PSR-4

- Organisation du projet avec namespaces App...
- Utilisation d'un autoloader PSR-4 (Composer ou maison)
- Structure conseillée : src/Entity, src/Service, src/Logger, public/index.php

Relations à faire apparaître

- Association : Utilisateur <=> ExempleLivres ; Livres <=> Auteur ; Livres <=> Catégorie
- Agrégation : Auteur <=> Livres ; Catégorie <=> Livres
- Composition : Bibliothèque <=> ExempleLivres

Scénario à implémenter dans index.php

1. Créer plusieurs auteurs, catégories, livres
2. Créer une bibliothèque
3. Ajouter les livres dans la bibliothèque
4. Afficher la liste de tous les livres disponibles
5. Rechercher des livres par auteur et par catégorie
6. Créer un utilisateur
7. Lui faire emprunter un livre (gestion d'exception si indisponible)
8. Afficher ses emprunts
9. Réafficher les livres disponibles

Contraintes

- Tous les attributs sont privés
- Getters/setters obligatoires
- Utiliser au minimum :
 - 1 interface
 - 1 trait
 - 1 classe abstraite
 - 1 méthode statique
 - 1 héritage
 - 1 composition
 - 1 agrégation
 - 1 association

- 1 exception personnalisée
- 1 enum
- 1 injection de dépendances
- Affichage propre (pas de var_dump brut)

Bonus : Évolution vers une base de données

1. Concevoir un schéma SQL (tables auteurs, catégories, livres, exemplaires, utilisateurs, emprunts)
2. Créer une couche repository (LivreRepository, AuteurRepository...)
3. Remplacer progressivement les collections internes par de la persistance via PDO